

```

1 // ----- Libraries -----
2
3 #include <Wire.h>           // For I2C communication (used by LCD)
4 #include <LiquidCrystal_I2C.h> // For controlling the I2C LCD
5 #include <TinyGPSPlus.h>    // For decoding GPS data
6 #include <SoftwareSerial.h> // For using extra RX/TX pins for GPS
7 #include <SPI.h>            // For SPI communication (used by SD card module)
8 #include <SD.h>             // For reading/writing files on an SD card
9 #include <TimeLib.h>        // For handling time and time zones
10
11 // ----- Pin Configuration -----
12 const int trigPin = 7;      // Ultrasonic sensor "Trigger" pin
13 const int echoPin = 8;      // Ultrasonic sensor "Echo" pin
14 const int chipSelectSD = 10; // Chip Select pin for SD card module
15 const int buttonPin = 4;    // Push button for manual logging
16 const int buzzerPin = 5;    // Buzzer pin for feedback beep
17
18 // ----- Objects -----
19 LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD object: I2C address=0x27, 16 columns, 2 rows
20 TinyGPSPlus gps;                  // GPS object to decode GPS data
21 SoftwareSerial gpsSerial(2, 3);    // Software serial: RX=2, TX=3 for GPS module
22 File logFile;                    // File object for SD card logging
23
24 // Time zone offset (example: India = UTC+5:30)
25 const long timeOffset = 5 * SECS_PER_HOUR + 30 * SECS_PER_MIN;
26
27 // ----- Variables -----
28 unsigned long lastUpdate = 0;    // Tracks when last update was done
29 long recordNumber = 1;          // Record counter
30 bool lastButtonState = HIGH;    // To detect button press/release
31
32 // ----- Functions -----

```

Fig. 3: Code snippet including the libraries

```

SD Write OK
System Initialized... Waiting for GPS fix
Record #1 , Dist: 8 cm , Lat, 17.873367 , Lng, 82.350349
SD Write OK
Record #2 , Dist: 0 cm , Lat, 17.873461 , Lng, 82.350266
SD Write OK

```

Fig. 4: Data logging output

The software workflow follows three stages: initialisation, continuous GPS monitoring, and button-triggered data logging. When the Arduino is powered on, it initialises all hardware components, including the LCD, SD card, and GPS module. After successful initialisation, the system enters the main loop, where it continuously waits for GPS data and monitors the push-button state. When the button is pressed, the system triggers a new log entry by measuring trench depth using the ultrasonic sensor, retrieving the latest GPS coordinates, displaying the information on the LCD, and storing the record on the SD card. A short buzzer beep confirms that the

data has been successfully saved. The record counter is then incremented, and the system returns to standby mode, ready for the next entry.

The software also follows a modular structure divided into user-defined functions for better readability and maintenance. The `getDistance()` function calculates distance using the ultrasonic sensor. The `beepBuzzer()` function provides audible confirmation, while `logToSD()` handles data storage on the SD card. The `makeLog()` function integrates all operations, including measurement, display, and data logging.

Program execution is controlled

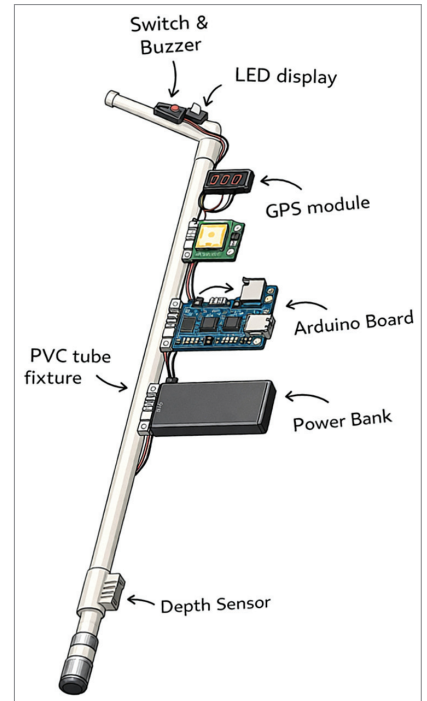


Fig. 5: Handheld frame of the portable ultrasonic-based depth and location logger for OFC laying operations

by the `setup()` and `loop()` functions. The `setup()` function initialises all components and displays system status, while the `loop()` function continuously reads GPS data, monitors the push button, and triggers logging when required, ensuring uninterrupted operation.

Data conversion and file logging

The depth measurement data is obtained in centimetres and combined with GPS coordinates formatted to six decimal places. These values are concatenated into a single line of text and stored in `GPSLOG.CSV` file on the SD card. The format of each record is shown in Fig. 4, which presents the serial monitor output of successful SD card logging, including recorded trench depth and corresponding GPS coordinates.

Construction and testing

Upload the source code to the Arduino Uno by selecting the correct board and COM port in the Arduino IDE, then proceed with